

HPC 2 - Design and implement Parallel Depth First Search based on existing algorithms using OpenMP. Use a Tree or an undirected graph for DFS.

```
#include <iostream>
#include <vector>
#include <queue>
#include <omp.h>
using namespace std;

class Graph {
public:
    int V;
    vector<vector<int>>> adj;
    vector<bool> visited;

    Graph(int V) {
        this->V = V;
        adj.resize(V);
        visited.resize(V, false);
    }

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void parallelDFS(int node) {
        #pragma omp critical
        {
            if (visited[node]) return;
            visited[node] = true;
            cout << node << " ";
        }

        #pragma omp parallel for
        for (int i = 0; i < adj[node].size(); i++) {
            int neighbor = adj[node][i];
            if (!visited[neighbor]) {
                parallelDFS(neighbor);
            }
        }
    }

};

int main() {
    int V, E;
    cout << "Enter number of vertices: ";
    cin >> V;
    Graph g(V);

    cout << "Enter number of edges: ";
    cin >> E;
```

```

cout << "Enter " << E << " edges (u v):\n";
for (int i = 0; i < E; i++) {
    int u, v;
    cin >> u >> v;
    g.addEdge(u, v);
}

int startNode;
cout << "Enter starting node for DFS: ";
cin >> startNode;
cout << "Parallel DFS: ";
g.parallelDFS(startNode);
cout << endl;

return 0;
}

```

o/p:

```

Enter number of vertices: 5
Enter number of edges: 4
Enter 4 edges (u v):
0 1
0 2
1 3
2 4
Enter starting node for DFS: 0
Parallel DFS: 0 1 3 2 4

```